Practical Scenario-Based Questions (Android Programming)

You are developing a **"Smart Utility Android Application"** for small business owners and general users. The app should include features like user data entry, communication, multimedia, event handling, and navigation support. Based on this scenario, perform the following tasks:

Questions:

1. **User Registration Interface (Replaced Question)**
   Design an Android application with a **user registration form** that collects Name, Email, and Password. Validate inputs and display a confirmation message upon successful submission.

1. Open **Android Studio**
2. Click **New Project**
3. Select **Empty Activity**
4. Name: UserRegistrationApp
5. Language: **Java** (or Kotlin if preferred)
6. Click **Finish**

Step 2: Design UI (XML Layout)

Go to:
🗁 res → layout → activity_main.xml

Replace code with this:

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout

    xmlns:android="http://schemas.android.com/apk/res/android"

    android:layout_width="match_parent"

    android:layout_height="match_parent"

    android:orientation="vertical"

    android:padding="20dp">
```

```
<EditText
    android:id="@+id/etName"
    android:hint="Enter Name"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"/>

<EditText
    android:id="@+id/etEmail"
    android:hint="Enter Email"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:inputType="textEmailAddress"/>

<EditText
    android:id="@+id/etPassword"
    android:hint="Enter Password"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:inputType="textPassword"/>

<Button
    android:id="@+id/btnRegister"
    android:text="Register"
```

```
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>

</LinearLayout>
```

Step 3: Write Backend Logic (Java Code)

Go to:
🗁 MainActivity.java

```
package com.example.userregistrationapp;

import android.os.Bundle;
import android.util.Patterns;
import android.view.View;
import android.widget.*;

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    EditText etName, etEmail, etPassword;
    Button btnRegister;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
```

```
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);

// Linking UI with code
etName = findViewById(R.id.etName);
etEmail = findViewById(R.id.etEmail);
etPassword = findViewById(R.id.etPassword);
btnRegister = findViewById(R.id.btnRegister);

btnRegister.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {

        String name = etName.getText().toString();
        String email = etEmail.getText().toString();
        String password = etPassword.getText().toString();

        // Validation
        if(name.isEmpty()) {
            etName.setError("Name is required");
            return;
        }
```

```
if(email.isEmpty()) {
    etEmail.setError("Email is required");
    return;
}

if(!Patterns.EMAIL_ADDRESS.matcher(email).matches()) {
    etEmail.setError("Enter valid email");
    return;
}

if(password.isEmpty()) {
    etPassword.setError("Password is required");
    return;
}

if(password.length() < 6) {
    etPassword.setError("Password must be at least 6 characters");
    return;
}

// Success Message
Toast.makeText(MainActivity.this,
        "Registration Successful!",
```

```
                Toast.LENGTH_LONG).show();

            }

        });

    }

}
```

2. **Restaurant Data Entry Form using Table Layout**
   Develop a **restaurant data entry form** using a Table Layout. Include fields such as Restaurant Name, Location, and Cuisine Type. Add radio buttons for selecting restaurant type (Fast Food, Fine Dining, Cafe).

1. Open **Android Studio**
2. Click **"New Project"**
3. Select **Empty Activity**
4. Click **Next**
5. Fill details:
   - Name → RestaurantFormApp
   - Language → **Java**
6. Click **Finish**
7. Wait until Gradle build completes

STEP 2: Go to Layout File

1. On left side, open:

app → res → layout → activity_main.xml

2. Switch to **Code view** (top right: Code / Split / Design → click Code)

STEP 3: Write TableLayout UI

👉 Delete all existing code and paste this:

```
<?xml version="1.0" encoding="utf-8"?>

<TableLayout

    xmlns:android="http://schemas.android.com/apk/res/android"

    android:layout_width="match_parent"
```

```
android:layout_height="match_parent"
android:stretchColumns="1"
android:padding="16dp">

<!-- Row 1: Restaurant Name -->
<TableRow>
    <TextView
        android:text="Restaurant Name:"
        android:padding="8dp"/>

    <EditText
        android:id="@+id/etName"
        android:hint="Enter Name"/>
</TableRow>

<!-- Row 2: Location -->
<TableRow>
    <TextView
        android:text="Location:"
        android:padding="8dp"/>

    <EditText
        android:id="@+id/etLocation"
```

```
        android:hint="Enter Location"/>
</TableRow>

<!-- Row 3: Cuisine -->
<TableRow>
    <TextView
        android:text="Cuisine Type:"
        android:padding="8dp"/>

    <EditText
        android:id="@+id/etCuisine"
        android:hint="Indian / Chinese"/>
</TableRow>

<!-- Row 4: Radio Buttons -->
<TableRow>
    <TextView
        android:text="Type:"
        android:padding="8dp"/>

    <RadioGroup
        android:id="@+id/radioGroup">
```

```
        <RadioButton
            android:id="@+id/rbFastFood"
            android:text="Fast Food"/>

        <RadioButton
            android:id="@+id/rbFineDining"
            android:text="Fine Dining"/>

        <RadioButton
            android:id="@+id/rbCafe"
            android:text="Cafe"/>
    </RadioGroup>
</TableRow>

<!-- Row 5: Button -->
<TableRow>
    <Button
        android:id="@+id/btnSubmit"
        android:text="Submit"
        android:layout_span="2"
        android:layout_gravity="center"/>
</TableRow>
```

```
</TableLayout>
```

STEP 4: Open Java File

1. Go to:

app → java → your package name → MainActivity.java

```
import android.os.Bundle;
import android.view.View;
import android.widget.*;

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    EditText etName, etLocation, etCuisine;
    RadioGroup radioGroup;
    Button btnSubmit;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        // Step 5.1: Connect UI with Java
```

```
etName = findViewById(R.id.etName);
etLocation = findViewById(R.id.etLocation);
etCuisine = findViewById(R.id.etCuisine);
radioGroup = findViewById(R.id.radioGroup);
btnSubmit = findViewById(R.id.btnSubmit);

// Step 5.2: Button Click Event
btnSubmit.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {

        // Step 5.3: Get Data
        String name = etName.getText().toString();
        String location = etLocation.getText().toString();
        String cuisine = etCuisine.getText().toString();

        // Step 5.4: Get Selected Radio Button
        int selectedId = radioGroup.getCheckedRadioButtonId();

        RadioButton selectedRadio = findViewById(selectedId);

        String type = "";
        if (selectedRadio != null) {
```

```
            type = selectedRadio.getText().toString();
        }

        // Step 5.5: Display Output
        String result = "Name: " + name +
            "\nLocation: " + location +
            "\nCuisine: " + cuisine +
            "\nType: " + type;

        Toast.makeText(MainActivity.this, result, Toast.LENGTH_LONG).show();
    }
  });
}
}
```

3. **Enhanced Form Validation**
   Modify the restaurant form to include **input validation** (e.g., empty fields, invalid entries) and display appropriate error messages.

□ STEP 1: Use Previous Project

👉 Open your **RestaurantFormApp** (from Question 2)

STEP 3: Open Java File

Go to:

app → java → your package → MainActivity.java

💻 STEP 4: Replace Button Logic with Validation Code

👉 Inside onCreate(), **replace only the button click code** with this:

```
btnSubmit.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {

        // Step 4.1: Get input values
        String name = etName.getText().toString().trim();
        String location = etLocation.getText().toString().trim();
        String cuisine = etCuisine.getText().toString().trim();

        // Step 4.2: Validation flag
        boolean isValid = true;

        // Step 4.3: Validate Name
        if (name.isEmpty()) {
            etName.setError("Restaurant name is required");
            isValid = false;
        }

        // Step 4.4: Validate Location
        if (location.isEmpty()) {
            etLocation.setError("Location is required");
            isValid = false;
```

```
}

// Step 4.5: Validate Cuisine
if (cuisine.isEmpty()) {
    etCuisine.setError("Cuisine type is required");
    isValid = false;
}

// Step 4.6: Validate Radio Button selection
int selectedId = radioGroup.getCheckedRadioButtonId();
if (selectedId == -1) {
    Toast.makeText(MainActivity.this,
            "Please select restaurant type",
            Toast.LENGTH_SHORT).show();
    isValid = false;
}

// Step 4.7: Stop if validation fails
if (!isValid) {
    return;
}

// Step 4.8: Get selected type
```

```
        RadioButton selectedRadio = findViewById(selectedId);

        String type = selectedRadio.getText().toString();


        // Step 4.9: Success Message

        String result = "Form Submitted Successfully!\n\n" +

                "Name: " + name +

                "\nLocation: " + location +

                "\nCuisine: " + cuisine +

                "\nType: " + type;


        Toast.makeText(MainActivity.this, result, Toast.LENGTH_LONG).show();

    }

});
```

4. **Contact Calling Application**
   Create an application that displays a list of contacts. When a user selects a contact and presses the **"Call" button**, it should initiate a phone call to that contact.

- Open **Android Studio**
- Click **New Project**
- Select **Empty Activity**
- Name → ContactCallApp
- Language → **Java**
- Click **Finish**

app → res → layout → activity_main.xml

STEP 3: Design UI

👉 Replace XML code with this:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <!-- Contact List -->
    <ListView
        android:id="@+id/listViewContacts"
        android:layout_width="match_parent"
        android:layout_height="300dp" />

    <!-- Call Button -->
    <Button
        android:id="@+id/btnCall"
        android:text="Call Selected Contact"
        android:layout_width="match_parent"
```

```
    android:layout_height="wrap_content"/>

</LinearLayout>
```

STEP 4: Add Permission in Manifest

Go to:

app → manifests → AndroidManifest.xml

👉 Add this line **outside <application> but inside <manifest>**:

```
<uses-permission android:name="android.permission.CALL_PHONE"/>
```

MainActivity.java

```
package com.example.contactcallapp;

import android.content.Intent;
import android.net.Uri;
import android.os.Bundle;
import android.view.View;
import android.widget.*;

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
```

```
ListView listView;
Button btnCall;

String[] contacts = {
        "Amit - 9876543210",
        "Rahul - 9123456780",
        "Priya - 9988776655"
};

String[] phoneNumbers = {
        "9876543210",
        "9123456780",
        "9988776655"
};

int selectedPosition = -1;

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
```

```
listView = findViewById(R.id.listViewContacts);
btnCall = findViewById(R.id.btnCall);

// Step 5.1: Set ListView Adapter
ArrayAdapter<String> adapter = new ArrayAdapter<>(
        this,
        android.R.layout.simple_list_item_single_choice,
        contacts
);

listView.setAdapter(adapter);
listView.setChoiceMode(ListView.CHOICE_MODE_SINGLE);

// Step 5.2: Handle item selection
listView.setOnItemClickListener((parent, view, position, id) -> {
    selectedPosition = position;
});

// Step 5.3: Call Button Click
btnCall.setOnClickListener(v -> {

    if (selectedPosition == -1) {
        Toast.makeText(MainActivity.this,
```

```
                "Please select a contact",
                Toast.LENGTH_SHORT).show();
            return;
        }

        String number = phoneNumbers[selectedPosition];

        // Step 5.4: Create Call Intent
        Intent intent = new Intent(Intent.ACTION_CALL);
        intent.setData(Uri.parse("tel:" + number));

        startActivity(intent);
    });
  }
}
```

5. **Permission Handling in Phone Call App**
   Enhance the phone call application to **request runtime permissions** for making calls and handle permission denial gracefully.

app → manifests → AndroidManifest.xml

👉 Ensure this line exists:

```
<uses-permission android:name="android.permission.CALL_PHONE"/>
```

STEP 3: Modify Java Code for Runtime Permission

Go to:

MainActivity.java

STEP 4: Add Required Imports

👉 Add these at the top:

```
import android.Manifest;
import android.content.pm.PackageManager;
import androidx.core.app.ActivityCompat;
import androidx.core.content.ContextCompat;
```

STEP 5: Create Permission Request Code

👉 Inside class (above onCreate()):

```
private static final int CALL_PERMISSION_CODE = 1;
```

STEP 6: Modify Call Button Logic

👉 Replace your call button code with this:

```
btnCall.setOnClickListener(v -> {


    if (selectedPosition == -1) {

        Toast.makeText(MainActivity.this,

                "Please select a contact",

                Toast.LENGTH_SHORT).show();

        return;

    }


    // Check permission

    if (ContextCompat.checkSelfPermission(

            MainActivity.this,

            Manifest.permission.CALL_PHONE)
```

```
            != PackageManager.PERMISSION_GRANTED) {

        // Request permission
        ActivityCompat.requestPermissions(
                MainActivity.this,
                new String[]{Manifest.permission.CALL_PHONE},
                CALL_PERMISSION_CODE);

    } else {
        // Permission already granted → make call
        makePhoneCall();
    }
});
```

STEP 7: Create Call Function

👉 Add this method inside class:

```
private void makePhoneCall() {

    String number = phoneNumbers[selectedPosition];

    Intent intent = new Intent(Intent.ACTION_CALL);
    intent.setData(Uri.parse("tel:" + number));

    startActivity(intent);
```

```
}
```

STEP 8: Handle Permission Result

👉 Add this method:

```
@Override
public void onRequestPermissionsResult(int requestCode,
                                       String[] permissions,
                                       int[] grantResults) {
    super.onRequestPermissionsResult(requestCode, permissions, grantResults);

    if (requestCode == CALL_PERMISSION_CODE) {

        if (grantResults.length > 0 &&
                grantResults[0] == PackageManager.PERMISSION_GRANTED) {

            Toast.makeText(this,
                    "Permission Granted",
                    Toast.LENGTH_SHORT).show();

            makePhoneCall();

        } else {

            Toast.makeText(this,
```

```
                "Permission Denied. Cannot make call.",

                Toast.LENGTH_LONG).show();

        }

    }

}
```

6. **Event Handling Demonstration**
   Develop an app that demonstrates **different user interaction events** such as button click, long press, and double tap. Display corresponding messages for each event.

1. Open **Android Studio**
2. Click **New Project**
3. Select **Empty Activity**
4. Name → EventHandlingApp
5. Language → **Java**
6. Click **Finish**

app → res → layout → activity_main.xml

STEP 3: Design UI

👉 Replace XML with this:

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout

    xmlns:android="http://schemas.android.com/apk/res/android"

    android:layout_width="match_parent"

    android:layout_height="match_parent"

    android:gravity="center"

    android:orientation="vertical">



    <Button
```

```
        android:id="@+id/btnClick"
        android:text="Click / Long Press"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>

    <TextView
        android:id="@+id/tvResult"
        android:text="Result will appear here"
        android:textSize="18sp"
        android:padding="20dp"/>

</LinearLayout>
```

STEP 4: Open Java File

Go to:

MainActivity.java

STEP 5: Write Event Handling Code

👉 Replace code with this:

```
package com.example.eventhandlingapp;

import android.os.Bundle;
import android.view.GestureDetector;
import android.view.MotionEvent;
import android.view.View;
```

```
import android.widget.*;

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    Button btnClick;
    TextView tvResult;
    GestureDetector gestureDetector;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        btnClick = findViewById(R.id.btnClick);
        tvResult = findViewById(R.id.tvResult);

        // STEP 5.1: Button Click Event
        btnClick.setOnClickListener(v -> {
            tvResult.setText("Button Clicked!");
        });
```

```
    // STEP 5.2: Long Press Event
    btnClick.setOnLongClickListener(v -> {
        tvResult.setText("Button Long Pressed!");
        return true; // Important (prevents click trigger)
    });

    // STEP 5.3: Double Tap Detection
    gestureDetector = new GestureDetector(this,
            new GestureDetector.SimpleOnGestureListener() {

                @Override
                public boolean onDoubleTap(MotionEvent e) {
                    tvResult.setText("Double Tap Detected!");
                    return true;
                }
            });

    // STEP 5.4: Attach gesture to screen
    findViewById(android.R.id.content)
            .setOnTouchListener((v, event) -> {
                return gestureDetector.onTouchEvent(event);
            });
}
```

}

7. **Gesture-Based Interaction**
   Extend the event handling app to include **swipe gestures** (left/right) and display different messages based on the swipe direction.

STEP 1: Use Previous Project

👉 Open your **EventHandlingApp** (from Question 6)

STEP 2: (Optional UI Update)

You can keep the same UI OR slightly update the text:

Go to:

activity_main.xml

Update TextView text:

```
<TextView
   android:id="@+id/tvResult"
   android:text="Perform Click / Long Press / Double Tap / Swipe"
   android:textSize="18sp"
   android:padding="20dp"/>
```

STEP 3: Open Java File

Go to:

MainActivity.java

STEP 4: Modify GestureDetector Code

👉 Replace ONLY the GestureDetector part with this updated version:

```
gestureDetector = new GestureDetector(this,

      new GestureDetector.SimpleOnGestureListener() {



          private static final int SWIPE_THRESHOLD = 100;
```

```
private static final int SWIPE_VELOCITY_THRESHOLD = 100;


@Override
public boolean onDoubleTap(MotionEvent e) {
    tvResult.setText("Double Tap Detected!");
    return true;
}


@Override
public boolean onFling(MotionEvent e1, MotionEvent e2,
                    float velocityX, float velocityY) {


    float diffX = e2.getX() - e1.getX();
    float diffY = e2.getY() - e1.getY();


    // Check horizontal swipe
    if (Math.abs(diffX) > Math.abs(diffY)) {


        if (Math.abs(diffX) > SWIPE_THRESHOLD &&
                Math.abs(velocityX) > SWIPE_VELOCITY_THRESHOLD) {


            if (diffX > 0) {
                // Swipe Right
```

```
                tvResult.setText("Swiped Right 👉");
            } else {
                // Swipe Left
                tvResult.setText("Swiped Left 👈");
            }
            return true;
        }
    }
    return false;
  }
});
```

STEP 5: Ensure Touch Listener is Attached

👉 Make sure this code is present (VERY IMPORTANT):

```
findViewById(android.R.id.content)
    .setOnTouchListener((v, event) -> {
        return gestureDetector.onTouchEvent(event);
    });
```

8. **Application Redirection to Map**
   Create a feature where clicking a **"View Location" button** redirects the user to a map application with a specified destination address.

▯ STEP 1: Create / Open Project

👉 You can:

- Use same project OR

- Create new → MapIntentApp

app → res → layout → activity_main.xml

STEP 3: Design UI

👉 Replace XML with this:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical">

    <EditText
        android:id="@+id/etLocation"
        android:hint="Enter location (e.g., Pune)"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:padding="10dp"/>

    <Button
        android:id="@+id/btnMap"
        android:text="View Location"
        android:layout_width="wrap_content"
```

```
        android:layout_height="wrap_content"/>


</LinearLayout>
```

STEP 4: Open Java File

Go to:

MainActivity.java

STEP 5: Write Code for Map Intent

👉 Replace code with this:

```
package com.example.mapintentapp;


import android.content.Intent;

import android.net.Uri;

import android.os.Bundle;

import android.view.View;

import android.widget.*;


import androidx.appcompat.app.AppCompatActivity;


public class MainActivity extends AppCompatActivity {


    EditText etLocation;

    Button btnMap;
```

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    etLocation = findViewById(R.id.etLocation);
    btnMap = findViewById(R.id.btnMap);

    btnMap.setOnClickListener(v -> {

        String location = etLocation.getText().toString().trim();

        if (location.isEmpty()) {
            etLocation.setError("Enter a location");
            return;
        }

        // Create Map Intent
        Uri uri = Uri.parse("geo:0,0?q=" + location);
        Intent intent = new Intent(Intent.ACTION_VIEW, uri);

        intent.setPackage("com.google.android.apps.maps");
```

```
            startActivity(intent);

        });

    }

}
```

9. **Implicit Intent Implementation**
   Implement **implicit intents** to open other applications such as browser, dialer, or map based on user actions.

STEP 1: Create / Open Project

👉 Create new project:

- Name → ImplicitIntentApp
- Language → **Java**

OR use previous project

STEP 2: Open Layout File

Go to:

app → res → layout → activity_main.xml

STEP 3: Design UI

👉 Replace XML with this:

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout

    xmlns:android="http://schemas.android.com/apk/res/android"

    android:layout_width="match_parent"

    android:layout_height="match_parent"

    android:gravity="center"

    android:orientation="vertical"
```

```
    android:padding="20dp">

    <Button
        android:id="@+id/btnBrowser"
        android:text="Open Browser"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>

    <Button
        android:id="@+id/btnDialer"
        android:text="Open Dialer"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>

    <Button
        android:id="@+id/btnMap"
        android:text="Open Map"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>

</LinearLayout>
```

STEP 4: Open Java File

Go to:

MainActivity.java

STEP 5: Write Java Code

👉 Replace code with this:

```
package com.example.implicitintentapp;


import android.content.Intent;

import android.net.Uri;

import android.os.Bundle;

import android.widget.Button;


import androidx.appcompat.app.AppCompatActivity;


public class MainActivity extends AppCompatActivity {


    Button btnBrowser, btnDialer, btnMap;


    @Override

    protected void onCreate(Bundle savedInstanceState) {

        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_main);


        btnBrowser = findViewById(R.id.btnBrowser);

        btnDialer = findViewById(R.id.btnDialer);
```

```
    btnMap = findViewById(R.id.btnMap);

    // STEP 5.1: Open Browser
    btnBrowser.setOnClickListener(v -> {
        Intent intent = new Intent(Intent.ACTION_VIEW);
        intent.setData(Uri.parse("https://www.google.com"));
        startActivity(intent);
    });

    // STEP 5.2: Open Dialer
    btnDialer.setOnClickListener(v -> {
        Intent intent = new Intent(Intent.ACTION_DIAL);
        intent.setData(Uri.parse("tel:9876543210"));
        startActivity(intent);
    });

    // STEP 5.3: Open Map
    btnMap.setOnClickListener(v -> {
        Uri uri = Uri.parse("geo:0,0?q=Pune");
        Intent intent = new Intent(Intent.ACTION_VIEW, uri);
        startActivity(intent);
    });
}
```

}

10. **Audio Player Application**
    Develop an audio player app with **Play, Pause, and Stop buttons** to play audio files stored on the device.

□ STEP 1: Create / Open Project

👉 Create new project:

- Name → AudioPlayerApp
- Language → **Java**

STEP 2: Add Audio File

👉 VERY IMPORTANT STEP ❗

1. Go to:

app → res

2. Right-click → **New** → **Android Resource Directory**
3. Select:

- Resource Type → **raw**
- Click **OK**

👉 A folder will be created:

res → raw

4. Copy any .mp3 file into this folder
   Example:

song.mp3

👉 Final path:

res/raw/song.mp3

STEP 3: Design UI

Go to:

res → layout → activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical">

    <Button
        android:id="@+id/btnPlay"
        android:text="Play"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>

    <Button
        android:id="@+id/btnPause"
        android:text="Pause"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>

    <Button
        android:id="@+id/btnStop"
        android:text="Stop"
```

```
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>

</LinearLayout>
```

STEP 4: Open Java File

Go to:

MainActivity.java

STEP 5: Write Audio Player Code

👉 Replace with this:

```
package com.example.audioplayerapp;

import android.media.MediaPlayer;
import android.os.Bundle;
import android.widget.Button;

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    Button btnPlay, btnPause, btnStop;
    MediaPlayer mediaPlayer;
```

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    btnPlay = findViewById(R.id.btnPlay);
    btnPause = findViewById(R.id.btnPause);
    btnStop = findViewById(R.id.btnStop);

    // STEP 5.1: Initialize MediaPlayer
    mediaPlayer = MediaPlayer.create(this, R.raw.song);

    // STEP 5.2: Play Button
    btnPlay.setOnClickListener(v -> {
        if (!mediaPlayer.isPlaying()) {
            mediaPlayer.start();
        }
    });

    // STEP 5.3: Pause Button
    btnPause.setOnClickListener(v -> {
        if (mediaPlayer.isPlaying()) {
            mediaPlayer.pause();
```

```
        }
    });

    // STEP 5.4: Stop Button
    btnStop.setOnClickListener(v -> {
        if (mediaPlayer.isPlaying()) {
            mediaPlayer.stop();

            // Reinitialize after stop
            mediaPlayer = MediaPlayer.create(this, R.raw.song);
        }
    });
}

@Override
protected void onDestroy() {
    super.onDestroy();

    // Release resources
    if (mediaPlayer != null) {
        mediaPlayer.release();
        mediaPlayer = null;
    }
```

```
    }

}
```

11.Audio Selection Feature
Enhance the audio player to allow users to **select audio files from device storage** and play them dynamically.

Multi-Feature Integration Enhancement

STEP 1: Use Previous Project

👉 Open your **AudioPlayerApp** (Question 10)

🎯 **WHAT WE WILL ADD**

✓ File picker
✓ Select .mp3 from phone
✓ Play selected file

STEP 2: Update Layout (Add Select Button)

Go to:

res → layout → activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout

    xmlns:android="http://schemas.android.com/apk/res/android"

    android:layout_width="match_parent"

    android:layout_height="match_parent"

    android:gravity="center"

    android:orientation="vertical">
```

```
<Button
    android:id="@+id/btnSelect"
    android:text="Select Audio"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"/>

<Button
    android:id="@+id/btnPlay"
    android:text="Play"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"/>

<Button
    android:id="@+id/btnPause"
    android:text="Pause"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"/>

<Button
    android:id="@+id/btnStop"
    android:text="Stop"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"/>
```

```
</LinearLayout>
```

MainActivity.java

💻 STEP 4: Add Required Imports

👉 Add these:

```
import android.content.Intent;
import android.media.MediaPlayer;
import android.net.Uri;
```

🔢 STEP 5: Declare Variables

👉 Inside class:

```
Button btnSelect, btnPlay, btnPause, btnStop;
MediaPlayer mediaPlayer;
Uri audioUri;

private static final int PICK_AUDIO = 1;
```

💻 STEP 6: Initialize Buttons

👉 Inside onCreate():

```
btnSelect = findViewById(R.id.btnSelect);
btnPlay = findViewById(R.id.btnPlay);
btnPause = findViewById(R.id.btnPause);
btnStop = findViewById(R.id.btnStop);
```

STEP 7: Add File Picker Logic

👉 Inside onCreate():

```
btnSelect.setOnClickListener(v -> {

    Intent intent = new Intent(Intent.ACTION_GET_CONTENT);
    intent.setType("audio/*");
    startActivityForResult(intent, PICK_AUDIO);
});
```

STEP 8: Handle Selected File

👉 Add this method:

```
@Override

protected void onActivityResult(int requestCode, int resultCode, Intent data) {

    super.onActivityResult(requestCode, resultCode, data);



    if (requestCode == PICK_AUDIO && resultCode == RESULT_OK && data != null) {



        audioUri = data.getData();



        Toast.makeText(this, "Audio Selected!", Toast.LENGTH_SHORT).show();



        // Prepare MediaPlayer

        mediaPlayer = MediaPlayer.create(this, audioUri);

    }

}
```

STEP 9: Update Play / Pause / Stop

👉 Add this inside onCreate():

```
// Play

btnPlay.setOnClickListener(v -> {
```

```
    if (mediaPlayer != null && !mediaPlayer.isPlaying()) {
        mediaPlayer.start();
    } else {
        Toast.makeText(this, "Select audio first", Toast.LENGTH_SHORT).show();
    }
});

// Pause
btnPause.setOnClickListener(v -> {
    if (mediaPlayer != null && mediaPlayer.isPlaying()) {
        mediaPlayer.pause();
    }
});

// Stop
btnStop.setOnClickListener(v -> {
    if (mediaPlayer != null) {
        mediaPlayer.stop();
        mediaPlayer.release();
        mediaPlayer = null;
    }
});
```

12.Extend the application to allow users to **select a contact, choose an audio file, and redirect to a location**, all through a well-designed interface with smooth navigation between activities.

STEP 1: Create New Project

1. Open **Android Studio**
2. Click **New Project**
3. Select **Empty Activity**
4. Name → SmartUtilityApp
5. Language → **Java**
6. Click **Finish**

STEP 2: Create New Activities

👉 Right-click package → **New** → **Activity** → **Empty Activity**

Create:

- CallActivity
- AudioMapActivity

STEP 3: Design MainActivity UI (Menu)

🗁 `activity_main.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical">

    <Button
        android:id="@+id/btnCallFeature"
        android:text="Contact Calling"
```

```
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>

    <Button
        android:id="@+id/btnAudioMap"
        android:text="Audio + Map Feature"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>

</LinearLayout>
```

STEP 4: MainActivity Logic

🗁 MainActivity.java

```
import android.content.Intent;
import android.os.Bundle;
import android.widget.Button;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    Button btnCallFeature, btnAudioMap;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        btnCallFeature = findViewById(R.id.btnCallFeature);
        btnAudioMap = findViewById(R.id.btnAudioMap);

        // Navigate to Call Feature
```

```
        btnCallFeature.setOnClickListener(v -> {
            startActivity(new Intent(this, CallActivity.class));
        });

        // Navigate to Audio + Map
        btnAudioMap.setOnClickListener(v -> {
            startActivity(new Intent(this, AudioMapActivity.class));
        });
    }
}
```

STEP 5: CallActivity UI

🗁 activity_call.xml

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:padding="16dp"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <ListView
        android:id="@+id/listViewContacts"
        android:layout_width="match_parent"
        android:layout_height="300dp"/>

    <Button
        android:id="@+id/btnCall"
        android:text="Call"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>
```

```
</LinearLayout>
```

STEP 6: CallActivity Logic

🗁 CallActivity.java

```
import android.content.Intent;
import android.net.Uri;
import android.os.Bundle;
import android.widget.*;
import androidx.appcompat.app.AppCompatActivity;

public class CallActivity extends AppCompatActivity {

    ListView listView;
    Button btnCall;

    String[] contacts = {"Amit - 9876543210", "Rahul - 9123456780"};
    String[] numbers = {"9876543210", "9123456780"};

    int selectedPosition = -1;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_call);

        listView = findViewById(R.id.listViewContacts);
        btnCall = findViewById(R.id.btnCall);

        ArrayAdapter<String> adapter = new ArrayAdapter<>(this,
                android.R.layout.simple_list_item_single_choice, contacts);
```

```
        listView.setAdapter(adapter);
        listView.setChoiceMode(ListView.CHOICE_MODE_SINGLE);

        listView.setOnItemClickListener((p, v, pos, id) -> selectedPosition = pos);

        btnCall.setOnClickListener(v -> {
            if (selectedPosition == -1) {
                Toast.makeText(this, "Select contact", Toast.LENGTH_SHORT).show();
                return;
            }

            String num = numbers[selectedPosition];

            Intent intent = new Intent(Intent.ACTION_DIAL);
            intent.setData(Uri.parse("tel:" + num));
            startActivity(intent);
        });
    }
}
```

STEP 7: AudioMapActivity UI

🗁 activity_audio_map.xml

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:padding="16dp"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <Button
        android:id="@+id/btnSelect"
```

```
        android:text="Select Audio"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>

    <Button
        android:id="@+id/btnPlay"
        android:text="Play"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>

    <EditText
        android:id="@+id/etLocation"
        android:hint="Enter location"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>

    <Button
        android:id="@+id/btnMap"
        android:text="Open Map"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>
</LinearLayout>
```

STEP 8: AudioMapActivity Logic

🗁 AudioMapActivity.java

```
import android.content.Intent;
import android.media.MediaPlayer;
import android.net.Uri;
import android.os.Bundle;
import android.widget.*;
```

```
import androidx.appcompat.app.AppCompatActivity;

public class AudioMapActivity extends AppCompatActivity {

    Button btnSelect, btnPlay, btnMap;
    EditText etLocation;
    MediaPlayer mediaPlayer;
    Uri audioUri;

    private static final int PICK_AUDIO = 1;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_audio_map);

        btnSelect = findViewById(R.id.btnSelect);
        btnPlay = findViewById(R.id.btnPlay);
        btnMap = findViewById(R.id.btnMap);
        etLocation = findViewById(R.id.etLocation);

        // Select Audio
        btnSelect.setOnClickListener(v -> {
            Intent intent = new Intent(Intent.ACTION_GET_CONTENT);
            intent.setType("audio/*");
            startActivityForResult(intent, PICK_AUDIO);
        });

        // Play Audio
        btnPlay.setOnClickListener(v -> {
```

```
        if (mediaPlayer != null) {
            mediaPlayer.start();
        } else {
            Toast.makeText(this, "Select audio first", Toast.LENGTH_SHORT).show();
        }
    });

    // Open Map
    btnMap.setOnClickListener(v -> {
        String loc = etLocation.getText().toString();

        if (loc.isEmpty()) {
            etLocation.setError("Enter location");
            return;
        }

        Uri uri = Uri.parse("geo:0,0?q=" + loc);
        startActivity(new Intent(Intent.ACTION_VIEW, uri));
    });
}

@Override
protected void onActivityResult(int req, int res, Intent data) {
    super.onActivityResult(req, res, data);

    if (req == PICK_AUDIO && res == RESULT_OK && data != null) {
        audioUri = data.getData();
        mediaPlayer = MediaPlayer.create(this, audioUri);
        Toast.makeText(this, "Audio Selected", Toast.LENGTH_SHORT).show();
    }
```

}

}